PayGuard AI

Project explainer for the team - how everything works, step by step

Status as of 9 November 2026 - Phases 1-8 complete, Phase 9 (submission) next

> **The one-sentence pitch**
>
> PayGuard stops risky PayPal payments BEFORE the money is captured, and explains why in plain language: a pre-capture risk gate that combines transaction and behaviour checks, a PayPal-USD (PYUSD) wallet-graph check, an XGBoost model with SHAP explanations, and a Gemini-written case report for human analysts.

What is in this document

1. What PayGuard does and its ground rules
2. The payment journey, step by step
3. The six layers of the system
4. How the risk score is calculated (with our three demo scenarios)
5. Machine learning: PaySim, XGBoost and SHAP
6. Blockchain intelligence: PYUSD wallet clusters
7. The Gemini investigation agent
8. Analyst review, step-up verification and the PDF case report
9. Security
10. The dashboard (what each tab shows)
11. Folder structure
12. How to run it on your laptop
13. Testing and live proof
14. Honest limitations
15. What is left to do
16. Glossary

1. What PayGuard does

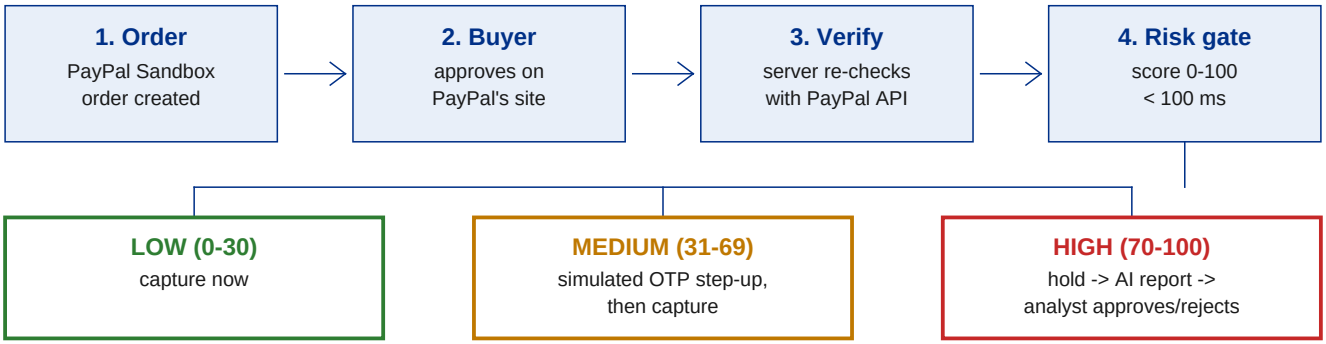
When someone pays through PayPal Checkout there are two moments: the buyer APPROVES the payment on PayPal's site, and later the merchant CAPTURES it (that is when money actually moves). PayGuard sits in between. After the buyer approves and before the merchant captures, PayGuard scores the payment and decides:

Risk band	Score	What happens
LOW	0-30	Capture straight away.
MEDIUM	31-69	Buyer must pass a step-up check (a simulated one-time code) before capture.
HIGH	70-100	Payment is held. An AI case report is written and a human analyst approves or rejects.

Ground rules we followed throughout

- PayPal Sandbox only - no real money, and we never claim to be PayPal's production fraud engine.
- Public and synthetic data only, and every piece of data is labelled as real or synthetic.
- Missing information is UNKNOWN, never 'safe'. Unknown can never produce a LOW (auto-capture) decision.
- The risk engine and the human analyst make decisions. Gemini only summarises evidence; it can never approve or reject.
- Wallet patterns are 'possible indicators' for investigation, never proof of fraud.
- No made-up numbers: every metric in our docs comes from a real run.

2. The payment journey, step by step



1. Create order: the dashboard (acting as the merchant's checkout) asks our backend to create a PayPal Sandbox order with intent CAPTURE. Nothing is charged yet.
2. Buyer approval: the buyer is sent to PayPal's own page, logs in with a Sandbox buyer account and clicks Continue.
3. Verify approval: PayPal sends the buyer back to our /payments/return link. We do NOT trust that link - the server asks PayPal's API directly and only continues if PayPal says APPROVED and the amount matches.
4. Risk analysis: the risk gate collects transaction, behaviour, wallet and model signals and produces a score in a few milliseconds (typically 3-20 ms).
5. Decision: the score maps to LOW / MEDIUM / HIGH and is stored against the order. A decision can be made only once per order, so nobody can 're-score' a HIGH order into a LOW one.
6. LOW: the merchant captures. MEDIUM: the buyer passes a one-time code first. HIGH: the payment is held.
7. For held payments, Gemini writes a case report in the background from the stored evidence.
8. An analyst (identified by their API key) approves or rejects with a written reason. Approve triggers the real capture through the same server-side gate; reject blocks capture permanently.

9. Every step is written to an audit log, and the analyst can download a PDF case report.

> **Key safety idea**
> Capture is decided only on the server, from the stored decision. The browser or caller cannot send a 'low score' or a 'force capture' flag - we removed that bypass in the first task. A capture can also only happen once: an atomic database claim plus PayPal's idempotency header stop double charging.

3. The six layers

Layer	What it does	Main code
1. Transaction intelligence	Amount vs the sender's average, cross-border / high-risk corridor, attempts in the last 24 h.	backend/app/transaction_intelligence/
2. Behavioural detection	Builds the sender's history on the server from captured payments: known devices, known recipients, average amount. Client-supplied history is ignored.	backend/app/behavioral_detection/
3. Blockchain intelligence	PYUSD transfer graph (NetworkX). Detects circular loops and pass-through fan-out, with exact evidence transfers and time windows.	backend/app/blockchain_intelligence/
4. Risk engine	XGBoost probability (trained on PaySim) + fixed rule points + wallet points = score. SHAP explains the model part.	backend/app/risk_engine/, ml/
5. AI investigation agent	Minimised evidence pack -> Gemini -> validated case report. Deterministic fallback if Gemini fails.	backend/app/investigation_agent/
6. PayPal integration + dashboard	Orders, approval, capture, webhooks; Streamlit dashboard for checkout and analysts.	backend/app/paypal_integration/, api/, dashboard/

Supporting pieces: FastAPI (the backend web server), SQLite (database: orders, transactions, alerts, step-up codes, events, audit log), and core/security.py (API keys).

4. How the risk score is calculated

The score is a simple, auditable sum - every point can be traced:

```
score = 5 (base)
      + model points   = XGBoost fraud probability x 100, capped at 40   (explained with SHAP)
      + rule points    = amount spike +15, unseen device +15, new recipient +10,
                         high-risk corridor +15, high velocity +10
                         (fixed policy weights, not learned)
      + wallet points  = circular loop +50, pass-through fan-out +40, allowlisted merchant -10
      -> clipped to 0..100
```

Then a safety policy: if any critical signal is missing (wallet data, sender history, or the ML model), a LOW result is raised to MEDIUM. The score itself is not changed; the dashboard shows a 'policy override' note instead.

Our three demo scenarios (real outputs)

Scenario	What it is	Score breakdown	Result
A. Normal payment	$45 to a familiar merchant, known device, allowlisted merchant wallet	5 + model 0 + rules 0 - wallet 10 = -5 -> 0	LOW -> captured
B. Unusual behaviour	$950 (11.9x the usual $80), new device, new recipient, no wallet	5 + model 10.8 + rules 40 = 55.8 -> 56	MEDIUM -> step-up
C. Suspicious ring	$4,800 to a wallet in a 5-wallet PYUSD loop, new device, offshore corridor	5 + model 6.7 + rules 55 + wallet 50 -> 100	HIGH -> hold + analyst

5. Machine learning: PaySim, XGBoost and SHAP

Data

PaySim (Kaggle 'ealaxi/paysim1', CC BY-SA 4.0) is 6.36 million SIMULATED mobile-money transactions - not PayPal data. All 8,213 fraud cases are TRANSFER or CASH_OUT, so we train on those 2.77 million rows.

Important decisions (and why)

- Balance columns are excluded: 97.8% of fraud rows have amount == starting balance (0.000% of normal rows), so they leak the answer - and PayPal does not give us a buyer's balance anyway.
- Only three features exist both in PaySim and in a live payment: payment amount, transaction type, and how many payments the recipient received before (counted only from EARLIER hours, so no future data leaks in).
- Time-based split: train on hours 1-400, tune on 401-500, test on 501-743 (data the model never saw).
- We did not invent synthetic device/country columns: the model would just learn back our own assumptions.

Honest results on the test period (124,630 rows, 2.13% fraud)

Metric	Value	Meaning
ROC-AUC	0.904	Good at ranking fraud above normal payments
PR-AUC	0.502	About 24x better than random (random = 0.021)
Precision / recall	62% / 34%	At the threshold chosen on validation data
False-positive rate	0.45%	Normal payments wrongly flagged

Plain summary: the model catches about a third of fraud with few false alarms, and its main signal is 'money going to a brand-new account'. That is why PayGuard combines it with rules and wallet intelligence.

SHAP

SHAP shows how much each feature pushed the model's answer up or down. Our SHAP values are real (a test checks they add up exactly to the model's output). Rule points are shown separately and are never called SHAP.

6. Blockchain intelligence (PYUSD)

PYUSD is PayPal's US-dollar stablecoin on Ethereum. A wallet is only linked to a payment when it is explicitly provided (PayPal data does not reveal wallets). The official contract 0x6c3ea9036406852006290770BEdFcAbA0e23A0e8 was verified against Paxos and PayPal documentation.

What is detected (computed from the transfers, not from labels)

- Circular loop: money goes around a cycle of 6 or fewer wallets within 72 hours (a layering indicator). Scenario C's ring: 5 wallets in 4.5 hours.
- Pass-through fan-out: a wallet sends to 4+ wallets within 24 hours AND passes on at least 80% of what it received, from at most 2 sources (a mule-dispersion indicator).
- Every finding carries its evidence: exact transfers, time window and data provenance. It is stored with the alert and shown in the dashboard and the PDF.

Real data and a lesson learned

With our Etherscan key we fetched 100 real PYUSD transfers (each with a real transaction hash). Our first fan-out rule flagged a real busy service wallet (24 payouts, but only 41% of incoming money passed through). We added the pass-through condition: 0 of 105 real wallets are flagged now, while the synthetic mule pattern still is. The demo story uses synthetic wallets (addresses 0xde5a...) so no real person's address is labelled in our video.

7. The Gemini investigation agent

- Evidence pack: only stored evidence, numbered E1, E2, ... No names, user IDs, devices, order IDs or wallet addresses are sent.
- Gemini (model gemini-3.5-flash) writes: a 1-2 sentence summary, 3 red flags each citing evidence IDs, and a suggested analyst action from a fixed list (escalate, verify with buyer, request info, manual review).
- Validation: the answer is rejected if it cites evidence that does not exist, has the wrong number of flags, or tries to approve/reject the payment.
- If Gemini is down or the answer is rejected, a deterministic summary quotes the stored evidence only (labelled 'no AI'). Temporary errors are retried up to 3 times.
- Runs in the background, so the payment decision is never delayed. Each report is audit-logged with the categories of evidence sent to Google.
- Note: on Google's free tier, Google may use what we send to improve its products - we send minimised, mostly synthetic evidence only.

8. Analyst review, step-up and the PDF case report

Step-up (MEDIUM risk)

A random 6-digit code valid for 5 minutes; only a keyed hash is stored. 3 wrong codes or 3 codes requested -> FAILED and escalated to an analyst. There is no SMS/email service, so in demo mode the code is shown on screen and labelled SIMULATED.

Analyst review (HIGH risk or failed step-up)

The analyst pastes their API key in the dashboard (their identity comes from the key), writes a reason and approves or rejects. Approve runs the real capture; reject blocks it forever. A decision can be made only once.

PDF case report

'PayGuard AI - Suspicious Activity Case Report': 10 sections (case IDs, score breakdown, transaction, behaviour, wallet evidence, SHAP, rule factors, AI report, analyst decision, audit trail). Built only from stored records. Every page says it is NOT a regulatory SAR filing. Each export is audit-logged.

9. Security

Who	Can do
Anyone	Health check, PayPal browser redirects, PayPal webhooks (signature-checked with PayPal), wallet lookups (demo/public data)
Merchant key	Create order, verify approval, risk analysis, step-up, capture
Merchant or analyst key	Read orders, scores, alerts, webhook events, download case reports
Analyst key only	Approve / reject held payments (identity taken from the key)

- Keys are long random values in .env, compared in constant time. If keys are missing, protected endpoints refuse everything.
- Webhooks are verified by PayPal's own API; duplicates are ignored; captures outside the gate are flagged.
- Strict input validation, CORS limited to the dashboard, security headers, and errors never reveal internals.
- With ENVIRONMENT=production the server refuses to start if demo mode is on or keys are missing.
- This is prototype-grade (static keys, unencrypted SQLite) - not production-certified.

10. The dashboard (http://localhost:8501)

Tab	What you see
PayPal Checkout + Risk Gate	Pick Scenario A/B/C -> create order -> approve on PayPal -> run risk analysis -> score breakdown, SHAP, rule factors -> capture / step-up / hold.
Interactive Graph Visualization	Wallet cluster graph (red = in a computed indicator, green = allowlisted merchant, grey = no indicator found), evidence table, and the real PYUSD data section.
Actionable AI Investigation Queue	Held cases with the Gemini report (red flags with evidence IDs), analyst key box, approve/reject with reason, PDF case report download.
PayPal Webhooks & Compliance	Sends a forged webhook to show it is rejected, and lists recent payment events.

11. Folder structure

Folder / file	Purpose
backend/app/main.py	Starts the FastAPI server, security middleware
backend/app/api/	Endpoints: payments.py, fraud.py (risk, alerts, reports, wallets), webhooks.py
backend/app/<layer folders>	The six layers described in section 3
backend/app/database.py	SQLite storage and audit log
backend/app/core/security.py	API keys and webhook/SSRF checks
dashboard/	Streamlit dashboard (app.py) and graph drawing (graph_viz.py)
ml/	PaySim features, XGBoost training, evaluation; trained model in ml/models/
data/	Synthetic demo wallets, synthetic behaviour seed, real PYUSD sample
scripts/fetch_pyusd_transfers.py	Fetches real PYUSD transfers from Etherscan
tests/	278 automated tests; docs/TESTING.md maps requirements to tests

12. How to run it on your laptop (Windows PowerShell)

One-time setup

```
cd "C:\path\to\SAFEGUARD PAYPAL"
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
Copy-Item .env.example .env      # then fill in your own keys (ask the team lead)
```

Keys needed in .env: PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET (Sandbox app), GEMINI_API_KEY, ETHERSCAN_API_KEY (optional), MERCHANT_API_KEY and ANALYST_API_KEYS (long random values). Never share .env or paste keys in chat.

Run (two PowerShell windows)

```
.\.venv\Scripts\python.exe -m uvicorn backend.app.main:app --port 8000
.\.venv\Scripts\python.exe -m streamlit run dashboard\app.py
```

Open http://localhost:8501. PayPal Sandbox tips (learned the hard way): create Sandbox accounts with country United States (India-to-India payments fail), and approve payments with the PERSONAL (buyer) account, not the Business one.

Tests

```
.\.venv\Scripts\python.exe -m pytest -q                          # offline, no network
$env:RUN_LIVE_TESTS = "1"; .\.venv\Scripts\python.exe -m pytest tests\test_live_services.py -v
```

13. Testing and live proof

Check	Result
Automated (offline) suite	278 passed, 5 live tests skipped by default; 91% line coverage
Full-stack test	Real backend + real dashboard: scenarios A, B, C, analyst approval, PDF - all pass
Live services	PayPal Sandbox order lifecycle, Gemini report, Etherscan PYUSD fetch - pass
Live Scenario A (manual)	LOW -> captured by PayPal Sandbox, capture 77Y92357DW632724T
Live Scenario C (manual)	HIGH -> held -> Gemini report -> analyst approved -> captured, capture 85Y243739J442351M (4,800.00 USD, confirmed COMPLETED via PayPal API)

14. Honest limitations (say these in the demo)

- Sandbox and synthetic data: model metrics come from PaySim simulated data, not real PayPal performance.
- The model has only 3 features; most of its signal is 'brand-new recipient account'.
- Sender IDs and device IDs are demo labels, not real identities or device fingerprints.
- Step-up codes are shown on screen (no SMS/email); in production they would be sent to the buyer.
- Wallet links are demo inputs; wallet patterns are leads, not proof; addresses are pseudonymous.
- Static API keys and an unencrypted SQLite database - fine for a prototype, not for production.
- Live PayPal webhook delivery is not yet tested (needs a public tunnel) - the handler logic is fully tested.

15. What is left to do (Phase 9)

1. Live PayPal webhook check: Cloudflare tunnel -> register webhook in the PayPal app -> PAYPAL_WEBHOOK_ID in .env -> one test payment.
2. GitHub: initialise the repository, choose a licence (MIT suggested), double-check .gitignore so .env and keys never get pushed.
3. Decide whether to commit the trained model and the real PYUSD sample so judges can run it without the 494 MB dataset.
4. Rewrite README.md to match what actually works (it still contains early claims), plus architecture docs.
5. Record the under-3-minute demo video (problem -> architecture -> Scenario A -> Scenario C hold, SHAP, Gemini, analyst -> limitations).
6. Read the hackathon rules (deadline, video length, repo requirements) and submit.

16. Glossary

Term	Meaning
Capture	The moment money actually moves after the buyer approved the payment
Sandbox	PayPal's test environment with fake money and test accounts
Step-up	An extra verification (here a one-time code) for medium-risk payments
PYUSD	PayPal USD, a US-dollar stablecoin on Ethereum
Layering / mule	Moving money through many accounts to hide where it came from
XGBoost	A machine-learning model made of many decision trees
SHAP	A method that shows how much each input pushed a model's prediction up or down
PR-AUC / ROC-AUC	Scores (0-1) for how well a model separates fraud from normal payments
Data leakage	When training data contains information that would not be available at prediction time
Webhook	A message PayPal sends to our server when something happens (e.g. capture completed)
SAR	Suspicious Activity Report - an official regulatory filing (our PDF is NOT one)